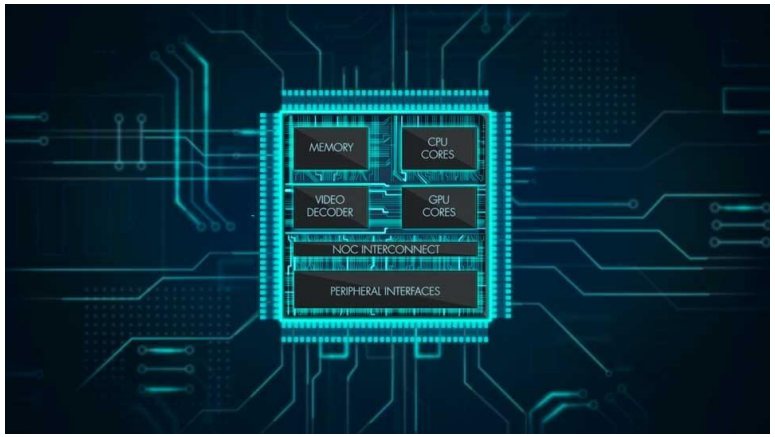
Digital System Design

Basic Practice 2



Date : 2026. 04. 02.

Instructor : **Prof. Hyun Kim**

Lecture Note : eclass.seoultech.ac.kr

Office : Mirae Hall #526

Web : idsl.seoultech.ac.kr

Contact : hyunkim@seoultech.ac.kr / 010-9600-5427

Contents

- Block Random Access Memory (BRAM)
- Problem 1: BRAM Write And Read
- Problem 2: BRAM Initialization
- Prob. 3: MAC with BRAM
- Appendix: BRAM Tiling and Usage

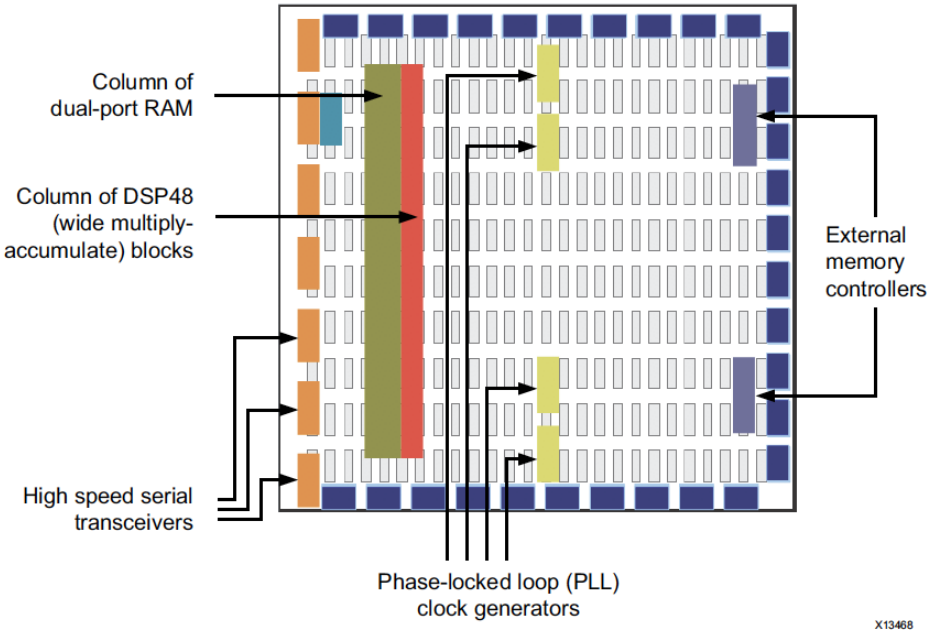
Contents

- Block Random Access Memory (BRAM)
- Problem 1: BRAM Write And Read
- Problem 2: BRAM Initialization
- Prob. 3: MAC with BRAM
- Appendix: BRAM Tiling and Usage

Block Random Access Memory

Block RAM (BRAM)

- Block RAM is a memory in FPGA fabric for **on-chip storage.**
- Available KV260 Block RAM Resources.



<BRAM layout in the FPGA fabric>

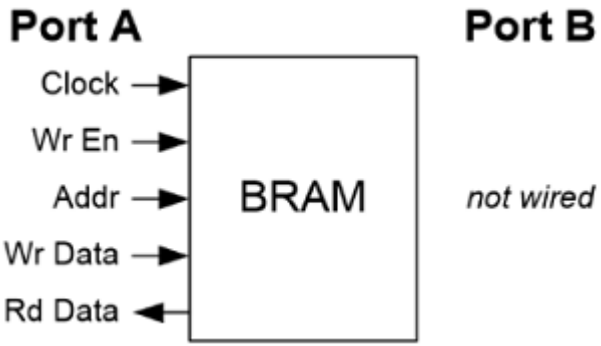
PARAMETER	KV260
DEVICE	Zynq™ UltraScale+™ MPSoC
FORM FACTOR	SOM + Carrier Card + Thermal Solution
STARTER KIT DIMENSIONS	119 mm x 140 mm x 36 mm
THERMAL COOLING SOLUTION	Active (Fan + Heatsink)
SYSTEM LOGIC CELLS	256K
BLOCK RAM BLOCKS	144
ULTRARAM BLOCKS	64
DSP SLICES	1.2K
ETHERNET INTERFACE	One 10/100/1000 Mb/s
DDR MEMORY	4 GB (4 x 512 Mb x 16-bit) [non-ECC]

<KV260 resources>

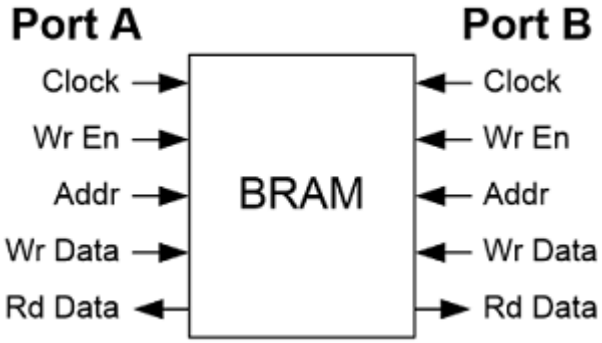
BRAM types

Key Features of BRAM

- **Flexible Port Modes:** Supports Single-Port and Dual-Port operations.
- **Block Capacity:** Available in 18Kb or 36Kb
- **Parallel Access:** Dual-port allows simultaneous rd/wr operations in a single clock cycle.



< Single Port BRAM >

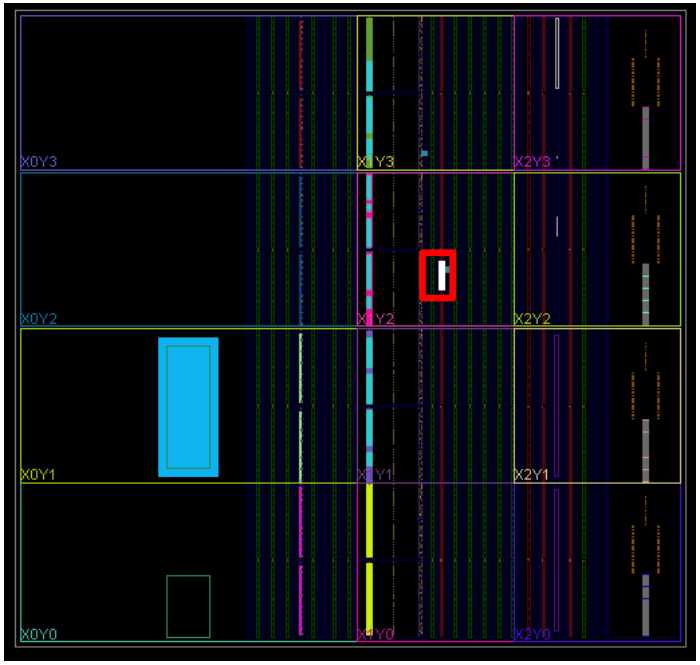


< Dual Port BRAM >

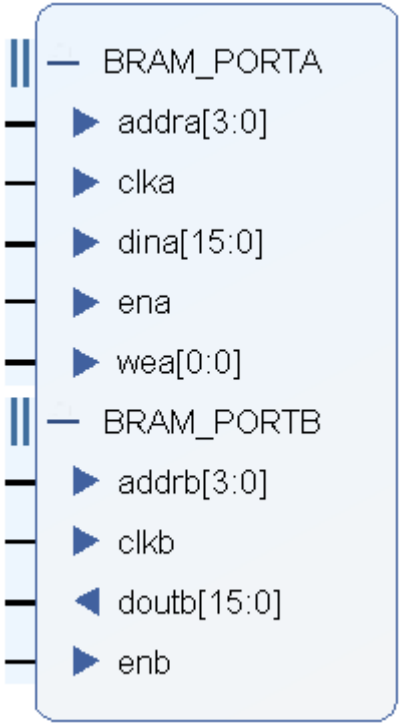
How to use a BRAM (1)

Two ways to implement BRAM in Vivado

- The first way is to **use Block Memory Generator IP** which is provided by Vivado.
- Another way is to **write a BRAM inferring code** so that the compiler can recognize that the code is for BRAM and implement it as BRAM.
- Both methods are useful, **but in this class we'll use the second method.**



< Implemented BRAM in the Vivado device layout >



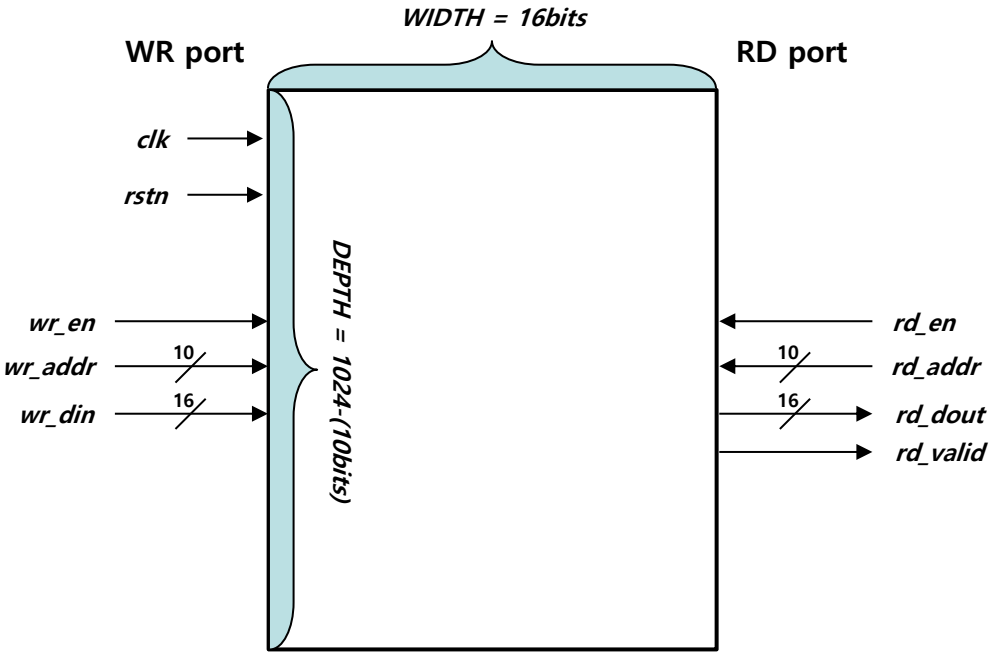
< Simple Dual Port BRAM IP >

How to use a BRAM (2)

Simple Dual Port BRAM inferring code

```
module simple_dual_port_bram #(
    parameter WIDTH      = 16,
    parameter DEPTH      = 1024,
    parameter ADDR_WIDTH = $clog2(DEPTH),
    parameter INIT_FILE  = ""
)(
    input  wire                clk,
    input  wire                rstn,
    input  wire                wr_en,
    input  wire                rd_en,
    input  wire [ADDR_WIDTH-1:0] wr_addr,
    input  wire [ADDR_WIDTH-1:0] rd_addr,
    input  wire [WIDTH-1:0]      wr_din,

    output reg                 rd_valid,
    output reg  [WIDTH-1:0]      rd_dout
);

    // BRAM
    (* ram_style = "block" *) reg [WIDTH-1:0] mem [0:DEPTH-1];
    // URAM
    // (* ram_style = "ultra" *) reg [WIDTH-1:0] mem [0:DEPTH-1];

    generate
        if (INIT_FILE != "") begin : use_init_file
            initial begin
                $readmemh(INIT_FILE, mem);
            end
        end else begin : init_to_zero
            integer i;
            initial begin
                for (i = 0; i < DEPTH; i = i + 1)
                    mem[i] = {WIDTH{1'b0}};
            end
        end
    endgenerate
```

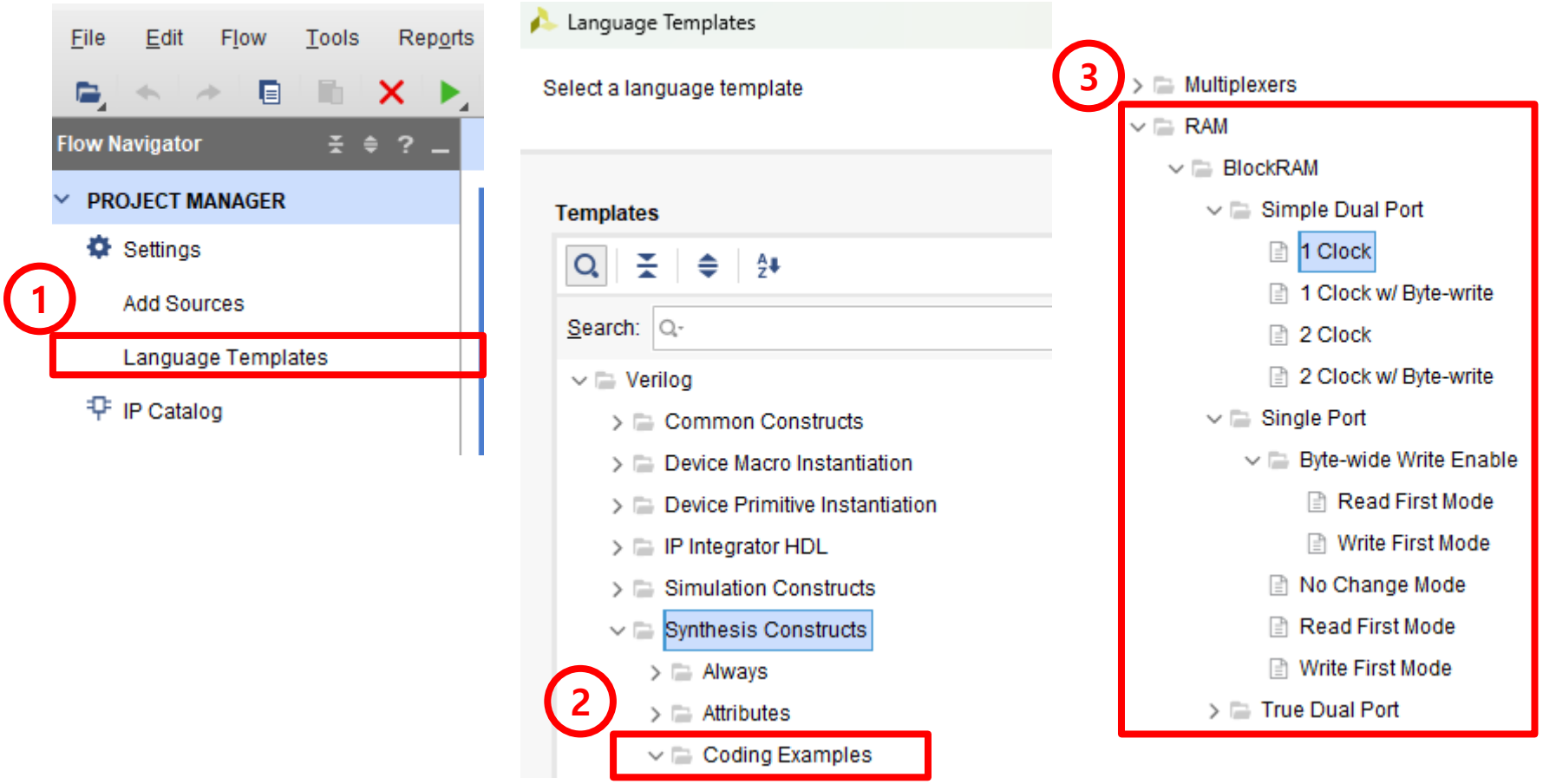
```
    always @(posedge clk) begin
        if (wr_en) mem[wr_addr] <= wr_din;
        if (rd_en) rd_dout <= mem[rd_addr]; // READ_FIRST
    end

    always @(posedge clk) begin
        if(!rstn) rd_valid <= 0;
        else rd_valid <= rd_en;
    end

endmodule
```



< Simple Dual Port BRAM >

How to use a BRAM (3)

Finding Vivado BRAM Inference Templates



Contents

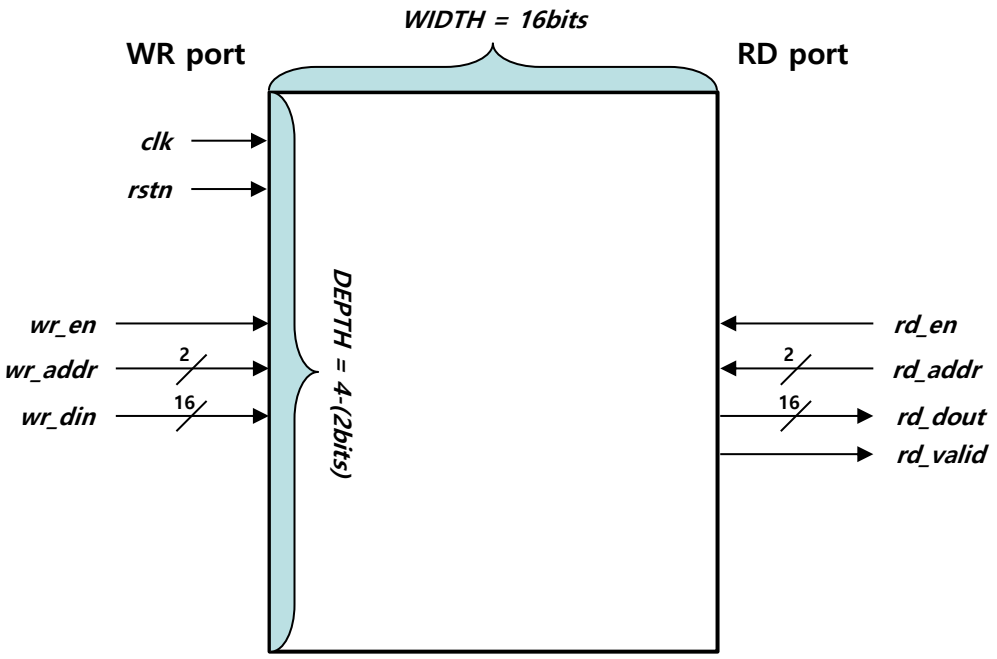
- Block Random Access Memory (BRAM)
- Problem 1: BRAM Write And Read
- Problem 2: BRAM Initialization
- Prob. 3: MAC with BRAM
- Appendix: BRAM Tiling and Usage

Prob. 1: BRAM Write And Read

BRAM write/read operation

- Write four 16-bit values to addresses 0 to 3 and read back the stored values.
- Verify that **"rd_dout"** matches the written data when **"rd_valid"** is asserted.

```
// Instantiation

simple_dual_port_bram #(
    .WIDTH(),
    .DEPTH(),
    .INIT_FILE()
)bram_inst(
    .clk(),
    .rstn(),
    .wr_en(),
    .rd_en(),
    .wr_addr(),
    .rd_addr(),
    .wr_din(),

    .rd_valid(),
    .rd_dout()
);
```

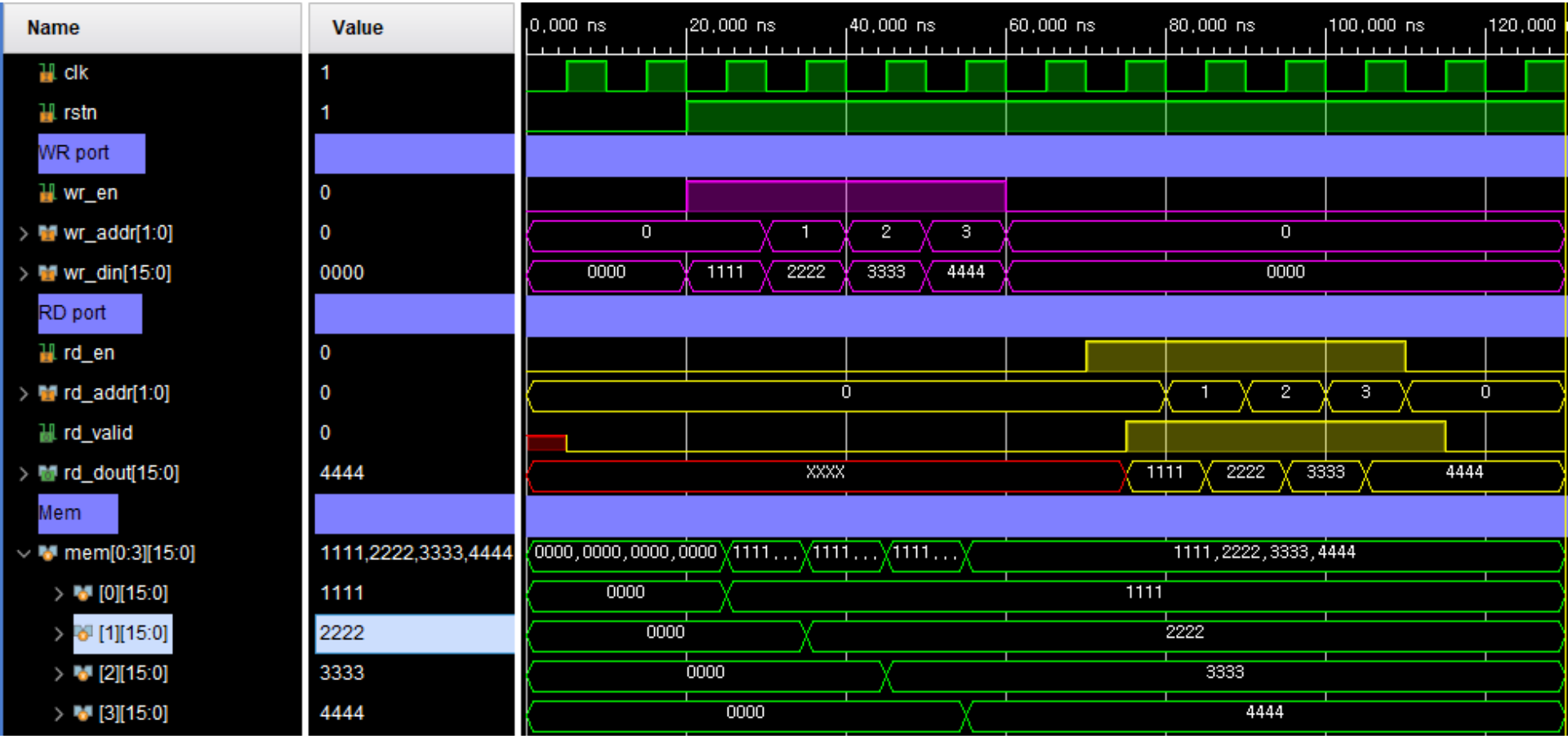


< Simple Dual Port Bram >

Prob. 1: Testbench

Simulation-level Verification Results

- Four 16-bit values are written to addresses 0 to 3.
- The read results match the written data, confirming correct BRAM operation.



Contents

- Block Random Access Memory (BRAM)
- Problem 1: BRAM Write And Read
- Problem 2: BRAM Initialization
- Prob. 3: MAC with BRAM
- Appendix: BRAM Tiling and Usage

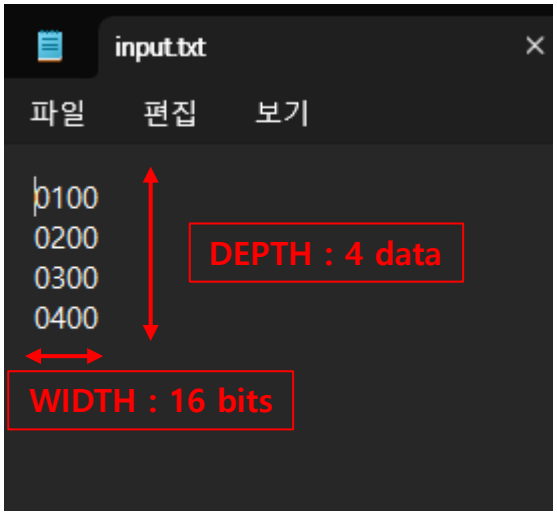
Prob. 2: BRAM Initialization

Initialize BRAM using a text file

- Use $readmemh to initialize BRAM from a text file.
- File and MemName are required arguments.
- Input and weight BRAMs load data from .txt files in address order.
- Create a test text file and set its path in the INIT_FILE parameter.
- In Windows file paths, **use ₩₩ instead of ₩.**
- After initialization, write 16'h8888 to address 3.
- Read addresses 0 to 3 to verify the updated BRAM contents.

```
simple_dual_port_bram #(
    .WIDTH(WIDTH),
    .DEPTH(DEPTH),
    .ADDR_WIDTH(ADDR_WIDTH),
    .INIT_FILE("D:₩₩DSDTA₩₩bram_test₩₩input.txt")
) bram_inst (
    .clk(clk),
    .rstn(rstn),
    .wr_en(wr_en),
    .rd_en(rd_en),
    .wr_addr(wr_addr),
    .rd_addr(rd_addr),
    .wr_din(wr_din),
    .rd_valid(rd_valid),
    .rd_dout(rd_dout)
);
```

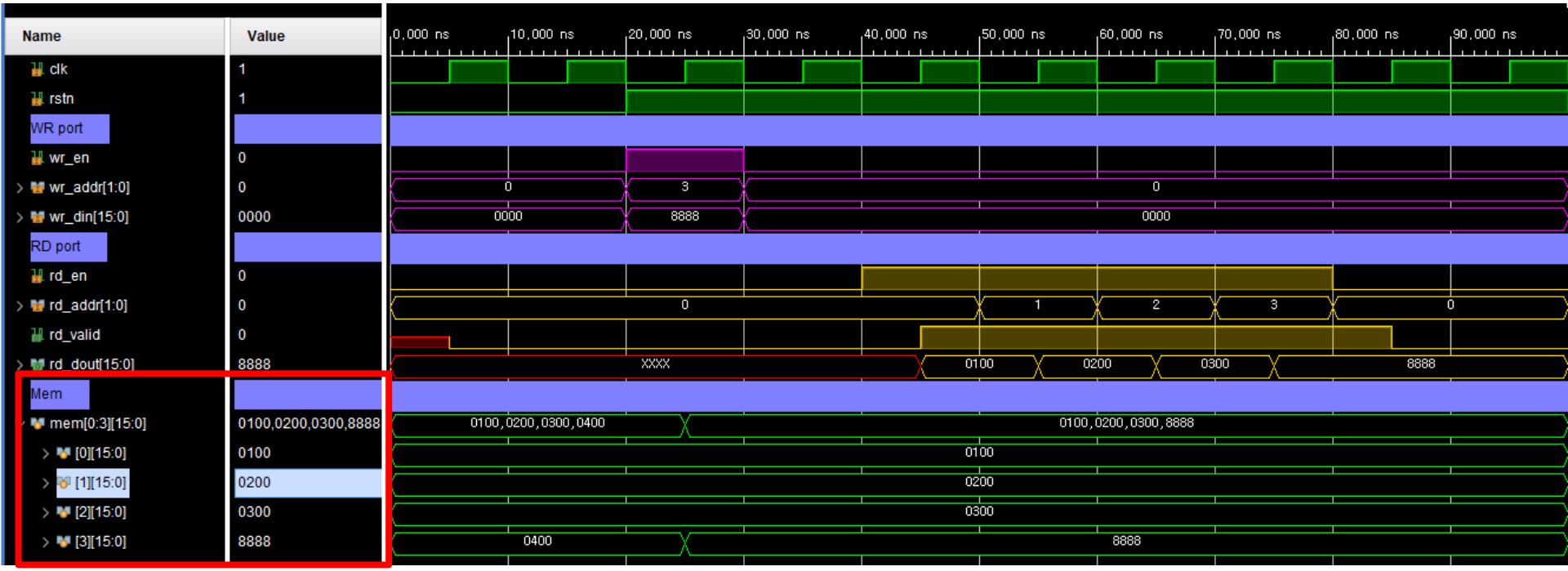
```
generate
    if (INIT_FILE != "") begin : use_init_file
        initial begin
            $readmemh(INIT_FILE, mem);
        end
    end else begin : init_to_zero
        integer i;
        initial begin
            for (i = 0; i < DEPTH; i = i + 1)
                mem[i] = {WIDTH{1'b0}};
        end
    end
endgenerate
```



Prob. 2: Testbench

Simulation-level Verification Results

- The BRAM is initialized exactly as defined in input.txt.
- The waveform confirms correct readback of the initialized data.



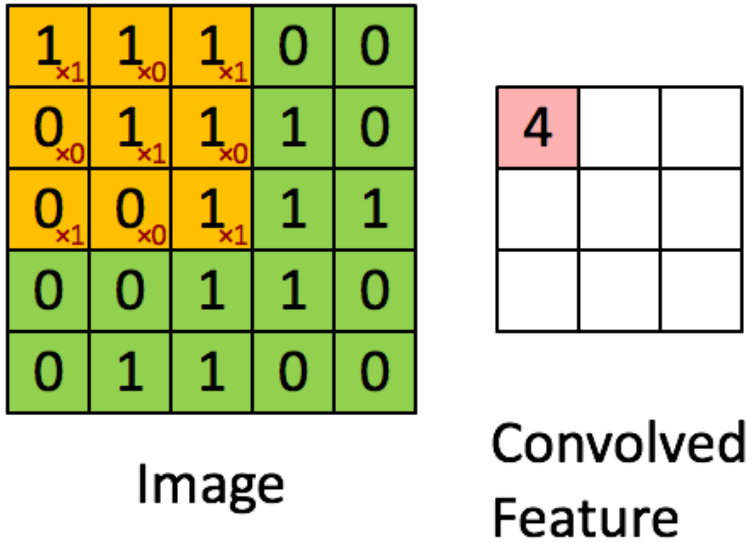
Contents

- Block Random Access Memory (BRAM)
- Problem 1: BRAM Write And Read
- Problem 2: BRAM Initialization
- Prob. 3: MAC with BRAM
- Appendix: BRAM Tiling and Usage

Prob. 3: MAC with BRAM (1)

Foundation of AI Operations

- **AI Models:** Weight files are mathematically applied to input data.
- **Convolution to MAC:** Complex algorithms break down into repeated MAC operations.
- **Vector Multiplication:** serves as the fundamental building block.

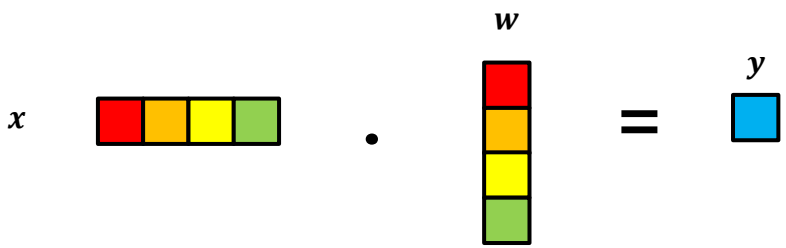


< Visualization of Convolution Operation Flow >

Prob. 3: MAC with BRAM (2)

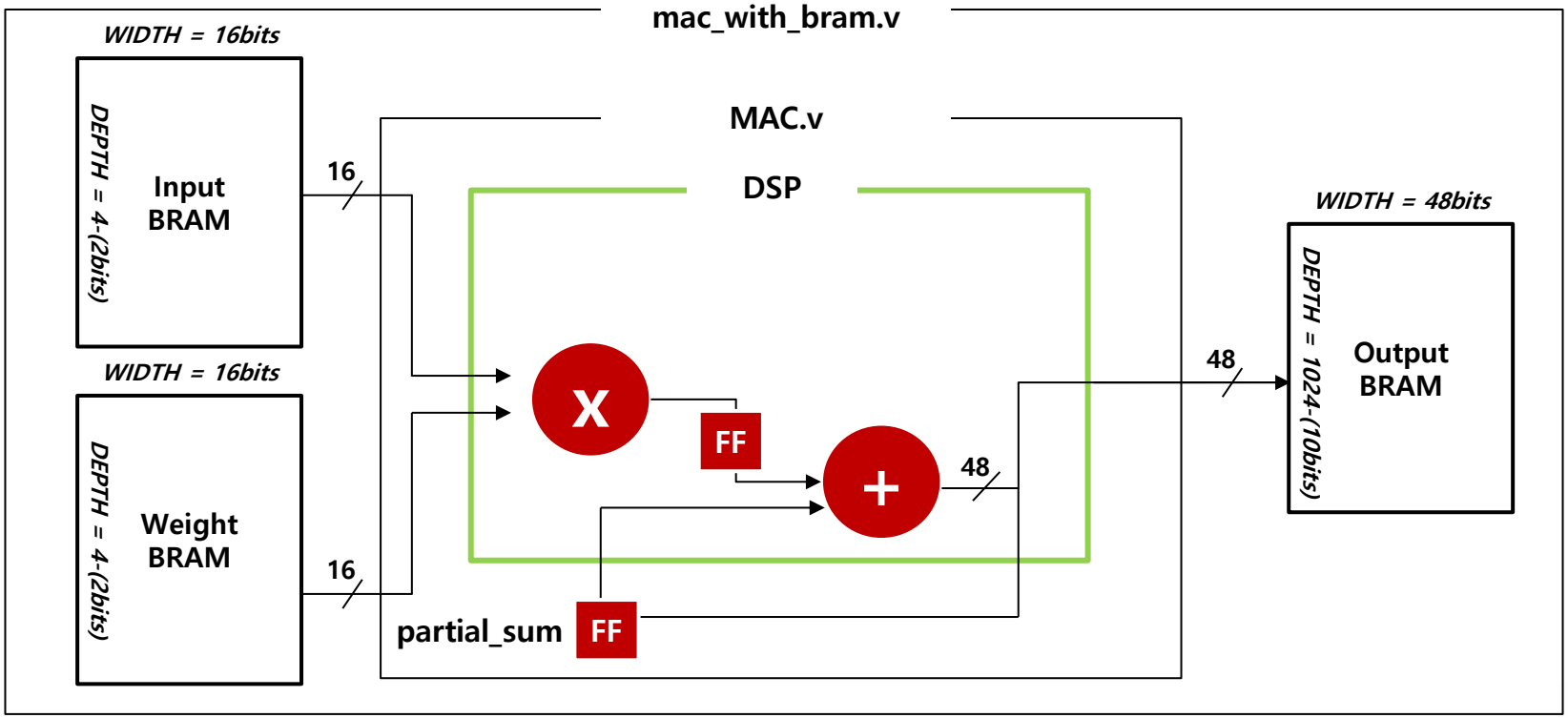
Computational Flow: Vector Multiplication as MAC

- **Concept:** Computing a single output via the dot product of two vectors.
- **Mathematical Proof:** $y = \sum (w_i \times x_i) = (w_1 \times x_1) + (w_2 \times x_2) + \ldots$
- **Hardware Mapping:**
  - **Multiply:** $w_i \times x_i$
  - **Accumulate:** $\sum$
  - **Conclusion:** One Vector Multiplication = A sequence of **MAC operations.**



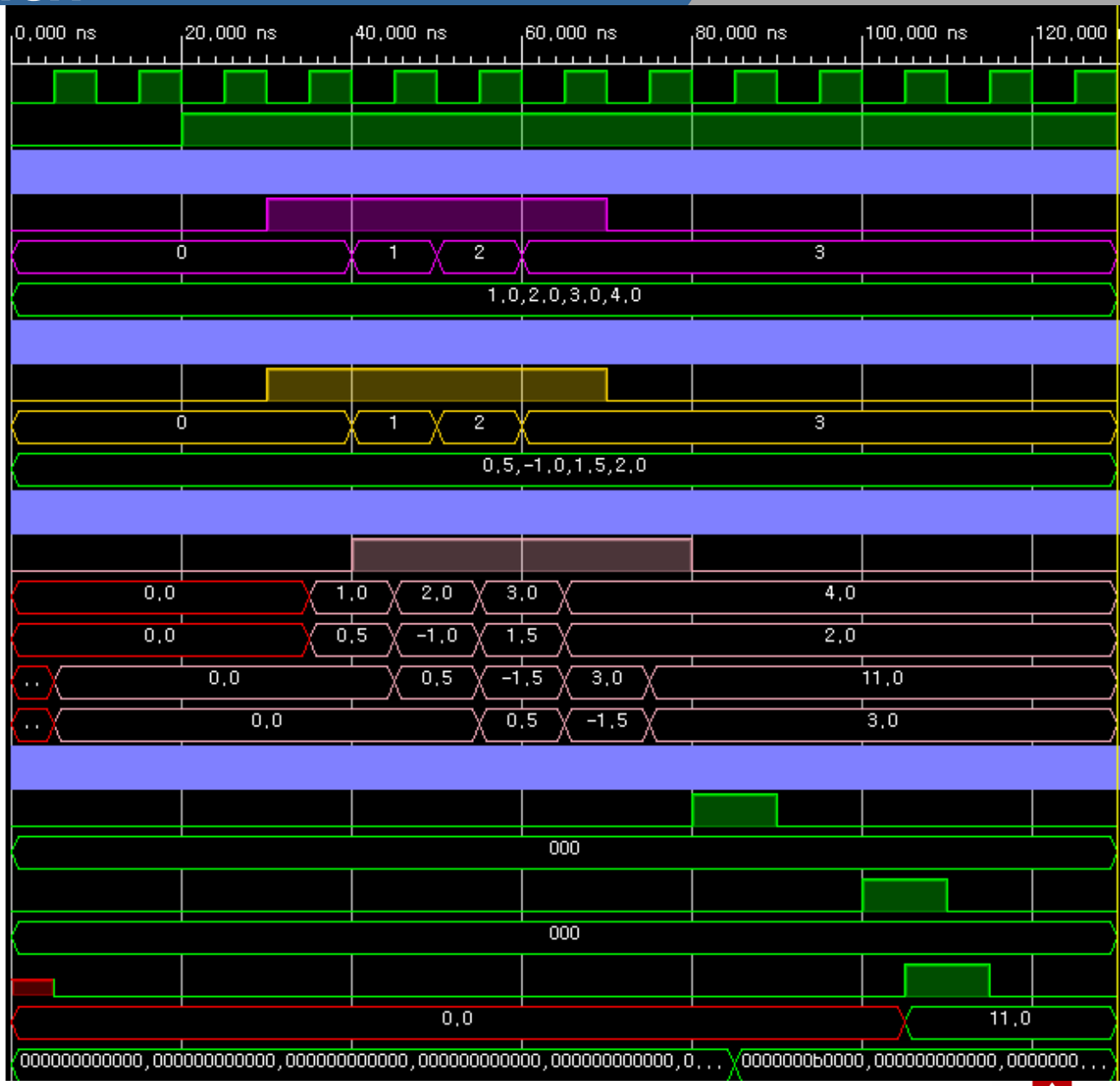
Prob. 3: MAC with BRAM (3)

- **Design a BRAM-based MAC module that reads input and weight data and stores the accumulated result.**
  - Reuse the previously designed simple dual-port BRAM module as-is.
  - Initialize the input and weight BRAMs using .txt files.
  - Read one pair of 16-bit input and weight values per cycle.
  - Perform multiplication and accumulation over four cycles.
  - Store the final MAC result in the output BRAM.



Prob. 3: Testbench

Name	Value
i_clk	1
i_rstn	1
Input BRAM	
i_input_rd_en	0
i_input_rd_addr[1:0]	3
mem[0:3][15:0]	1.0,2.0,3.0,4.0
weight BRAM	
i_weight_rd_en	0
i_weight_rd_addr[1:0]	3
mem[0:3][15:0]	0.5,-1.0,1.5,2.0
MAC	
i_dsp_enable	0
i_dsp_input[15:0]	4.0
i_dsp_weight[15:0]	2.0
o_dsp_output[47:0]	11.0
partial_sum[47:0]	3.0
output BRAM	
i_output_wr_en	0
i_output_wr_addr[9:0]	000
i_output_rd_en	0
i_output_rd_addr[9:0]	000
o_output_rd_valid	0
o_output_rd_dout[47:0]	11.0
mem[0:1023][47:0]	0000000b0000,00000



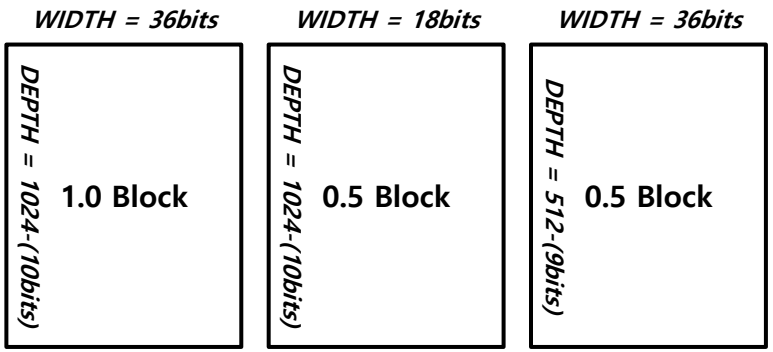
Contents

- Block Random Access Memory (BRAM)
- Problem 1: BRAM Write And Read
- Problem 2: BRAM Initialization
- Prob. 3: MAC with BRAM
- Appendix: BRAM Tiling and Usage

BRAM Tiling and Usage

BRAM Usage in Tiled Memory Mapping

- 1.0 BRAM = one BRAM36 = 1K × 36
- 0.5 BRAM = one BRAM18 = 1K × 18
- Prob. 1: 16 × 4 → 0.5 BRAM
- Prob. 3:
  - input 0.5 + weight 0.5 = 1.0 BRAM
  - Output: 48 × 1024 = 36(1.0) + 12(0.5) → 1.5 BRAMs
- Total in Prob. 3 = 2.5 BRAMs
- BRAM resources can be flexibly **split or combined** to match the required memory width and depth.



Prob 1. Synthesis

Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF	BRAM
synth_1 (active)	constrs_1	synth_design Complete!												0	1	0.5

Prob 3. Synthesis

Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF	BRAM
synth_1 (active)	constrs_1	synth_design Complete!												1	49	2.5

BRAM Tiling Experiment

Synthesize different memory sizes and compare BRAM usage

- Change the memory width and depth and run synthesis.
- Check how BRAM usage changes with each memory size.
- Observe when BRAM is split into 0.5 blocks or combined into larger memories.
- Compare the synthesis results for the following cases:

Memory Size	Expected BRAM Usage
18 × 1024	0.5
17 × 1024	0.5
19 × 1024	1.0
18 × 1025	1.0
36 × 512	0.5
1000 × 1024	28

Thank You!